

ТЕМА 2

ІТЕРАЦІЙНІ МЕТОДИ РОЗВ'ЯЗАННЯ НЕЛІНІЙНИХ РІВНЯНЬ І СИСТЕМ

Ітераційні методи розв'язання нелінійних рівнянь

Знаходження коренів нелінійного рівняння

$$f(x) = 0 \quad (3.1)$$

здійснюється у два етапи.

На першому етапі досліджується розташування коренів (у загальному випадку на комплексній площині) і визначаються області, в яких існує тільки один корінь і його кратність. Це дає нам початкове наближення до вибраного кореня.

На другому етапі в залежності від характеристик функції $f(x)$ будується певний ітераційний процес (ІП) уточнення кореня.

У загальному вигляді ІП можна записати так

$$x^{(k+1)} = \Phi_{(k)}(x^{(k)}, x^{(k-1)}, \dots, x^{(k-m+1)}), k = 0, 1, \dots \quad (3.2)$$

Для характеристики ІП (3.2) вводять такі поняття:

- *p-й порядок методу* – якщо виконується рівність $r^{(k+1)} = C \cdot (r^{(k)})^p$, де $C = \text{const}$, $r^{(k)} = |x_* - x^{(k)}|$ – похибка, а x_* – точний розв'язок (3.1) ($f(x_*) \equiv 0$);
- *стаціонарний метод* – якщо $\Phi_{(k)}$ не залежить від k , інакше – *нестационарний*;
- *m-кроковий* ($m \geq 1$) – якщо для отримання чергового елемента послідовності використовують m попередніх.

Метод дихотомії Якщо для рівняння (3.1) на відрізку $[a, b]$ ($a < b$) виконується умова $f(a) \cdot f(b) \leq 0$, то за допомогою процесу поділу цього відрізка пополам :

$$l = a; r = b; k = 0$$

$$\begin{cases} x = 0.5(l + r) \\ f(l) \cdot f(x) \leq 0 \mapsto r = x \\ f(l) \cdot f(x) > 0 \mapsto l = x \\ k = 1, 2, \dots, (|r - l| \leq \varepsilon) \end{cases}$$

$$x_* \cong 0.5(l + r)$$

знайдемо деякий дійсний корінь x_* з заданою точністю $0 < \varepsilon < 1$. Метод завжди збігається, але не може бути поширений для знаходження комплексних коренів і систем нелінійних рівнянь.

Метод простої ітерації (МПІ) Спочатку рівняння (3.1) приводиться до еквівалентного

$$x = s(x) \quad (3.3)$$

і ІП проводиться за правилом

$$x^{(k+1)} = s(x^{(k)}), \quad k = 0, 1, \dots \quad (3.4)$$

Функція $s(x)$ зазвичай вибирається у вигляді

$$s(x) = x + \tau(x)f(x), \quad (3.5)$$

причому функція $\tau(x)$ не повинна змінювати знак в області пошуку кореня $U_r(x_*) =$

$\{x: |x - x_*| \leq r\}$, а для $s(x)$ має виконуватись умова $|s'(x)| \leq q < 1$ в області $U_r(x_*)$.

Якщо $\tau(x) = \tau = const$, то отримаємо метод релаксації:

$$x^{(k+1)} = x^{(k)} + \tau \cdot f(x^{(k)}), \quad (3.6)$$

який збігається при умові $-2 < \tau \cdot f'(x_*) < 0$.

Якщо в області $U_r(x_*)$ виконується умови $f'(x) < 0$ & $0 < m_1 < |f'(x)| < M_1$, то (3.6) збігається при значеннях $\tau \in (0, 2/M_1)$ і для оптимального значення маємо $\tau_{opt} = 2/(m_1 + M_1)$.

Враховуючи, що для збіжного ІП мають місце нерівності:

$$|x_* - x^{(k)}| \leq \frac{q}{1-q} |x^{(k)} - x^{(k-1)}|, \quad |x_* - x^{(k)}| \leq \frac{q^k}{1-q} |x^{(1)} - x^{(0)}|,$$

то спрощену умову закінчення ІП вибирають у вигляді $|x^{(k+1)} - x^{(k)}| \leq \varepsilon$, $0 < \varepsilon < 1$.

Ітераційні процеси (3.4), (3.6) мають перший порядок (збігаються лінійно зі швидкістю геометричної прогресії зі знаменником q).

На основі лінійного ІП можна побудувати нестационарний ІП, який має квадратичну збіжність. Це так званий **процес Ейткена-Стеффенсена**. Його алгоритм: за відомим $x^{(k)}$ циклічно $k = 0, 3, 6, \dots$, $|x^{(k+3)} - x^{(k)}| \leq \varepsilon$, виконуємо наступні розрахунки

$$\begin{cases} x^{(k+1)} = s(x^{(k)}), & x^{(k+2)} = s(x^{(k+1)}), \\ x^{(k+3)} = x^{(k)} - \frac{(x^{(k)} - x^{(k+1)})^2}{x^{(k)} - 2x^{(k+1)} + x^{(k+2)}}. \end{cases} \quad (3.6)$$

Метод Ньютона (лінеаризації) Нехай функція $f(x)$ рівняння (3.1) двічі неперервно диференційована ($f(x) \in C^2(\Omega)$, $\Omega \in \mathbb{R}$), тоді

$$0 = f(x_*) = f(x^{(k)} + (x_* - x^{(k)})) = f(x^{(k)}) + (x_* - x^{(k)}) \cdot f'(\xi).$$

Звідси маємо $x_* = x^{(k)} - \frac{f(x^{(k)})}{f'(\xi)}$ і, замінивши невідоме значення ξ на відоме $x^{(k)}$,

отримаємо розрахункову формулу ІП Ньютона:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}, \quad (3.7)$$

Теор.2.1 Метод Ньютона збігається до кореня (3.1) при $\forall x^{(0)}$, якщо виконується умова $|f(x) \cdot f''(x)| < (f'(x))^2$.

Дов. Представимо (3.7) у формі МП $x = s(x) \equiv x - \frac{f(x)}{f'(x)}$. Знайдемо $s'(x)$:

$$s'(x) = 1 - \frac{(f')^2 - f \cdot f''}{(f')^2} = \frac{f \cdot f''}{(f')^2} = q \quad \left[\frac{f \cdot f''}{(f')^2} \Big|_{(x=x_*)} = 0 \right].$$

Якщо скористатись зауваженням до Теор.2.3, то для збіжності (3.7) необхідно і достатньо $|q| < 1$, а в нашому випадку $q=0$, отже теорема доведена.

Теор.2.2 Якщо $f'(x)$ і $f''(x)$ зберігають знак на $\Omega = U_r(x_*)$, а також виконується умова $f(x^{(0)}) \cdot f''(x^{(0)}) \geq 0$, то ІП (3.7) збігається до кореня x_* монотонно і для похибки маємо нерівність

$$|x_* - x^{(k)}| \leq \frac{M_2}{2m_1} |x^{(k)} - x^{(k-1)}|^2, \quad m_1 = \min_{\Omega} |f'(x)|, \quad M_2 = \max_{\Omega} |f''(x)|,$$

а ІП (3.7) має 2-й порядок.

Дов. Монотонність: Маємо ланцюжок рівностей

$$x^{(k+1)} - x^{(k)} = s(x^{(k)}) - s(x^{(k-1)}) = s'(\zeta)(x^{(k)} - x^{(k-1)})$$

і, враховуючи умови теореми про знак $f'(x)$, $f''(x)$ отримаємо $\text{sign}(s'(\zeta)) = 1$, а отже при $x^{(k)} < x^{(k-1)} \Rightarrow x^{(k+1)} < x^{(k)}$.

Оцінка порядку: Маємо ланцюжок рівностей

$$\begin{aligned} f(x^{(k)}) &= f(x^{(k)} \pm x^{(k-1)}) = f(x^{(k-1)} + (x^{(k)} - x^{(k-1)})) = \\ &= \underbrace{f(x^{(k-1)}) + (x^{(k)} - x^{(k-1)}) \frac{f'(x^{(k-1)})}{1!} + \frac{f''(\zeta)}{2!} (x^{(k)} - x^{(k-1)})^2}_{=0} = \\ &= \frac{f''(\zeta)}{2!} (x^{(k)} - x^{(k-1)})^2; \\ x_* - x^{(k)} &= -\frac{f(x^{(k)})}{f'(\xi)} = -\frac{1}{2} \frac{f''(\zeta)}{f'(\xi)} (x^{(k)} - x^{(k-1)})^2. \end{aligned} \quad (*)$$

Звідси отримаємо таку нерівність (можна використати для закінчення ІІ !):

$$|x_* - x^{(k)}| \leq \frac{M_2}{2m_1} |x^{(k)} - x^{(k-1)}|^2.$$

Замінімо в (*) $x^{(k)}$ на x_* :

$$\begin{aligned} 0 &= f(x_*) = f(x^{(k-1)} + (x_* - x^{(k-1)})) = \frac{f'(x^{(k-1)})}{1!} (x_* - x^{(k-1)}) + \frac{f''(\zeta_1)}{2!} (x_* - x^{(k-1)})^2; \\ x_* - x^{(k-1)} &+ \underbrace{\frac{f(x^{(k-1)})}{f'(x^{(k-1)})}}_{x^{(k)}} = -\frac{f''(\zeta_1)}{2f'(x^{(k-1)})} (x_* - x^{(k-1)})^2; \\ x_* - x^{(k)} &= -\frac{f''(\zeta_1)}{2f'(x^{(k-1)})} (x_* - x^{(k-1)})^2. \end{aligned}$$

З останньої рівності, з урахуванням означення порядку ІІ, маємо:

$$r^{(k+1)} = C \cdot (r^{(k)})^2, \quad C = \frac{M_2}{2m_1} \in \text{const},$$

Теорема повністю доведена.

Виходячи із геометричних побудов, ІІ (3.7) має назву **методу дотичних**.

Для кореня x_* кратності n , ІІ (3.7) записують у формі

$$x^{(k+1)} = x^{(k)} - n * \frac{f(x^{(k)})}{f'(x^{(k)})}$$

При заміні у (3.7) похідної на поділену різницю, отримаємо **двокрокові** ІІ, зокрема

А) метод січних :

$$x^{(k+1)} = x^{(k)} - \frac{(x^{(k)} - x^{(k-1)})f(x^{(k)})}{f(x^{(k)}) - f(x^{(k-1)})} \quad (3.8)$$

Б) метод Курчатова :

$$x^{(k+1)} = x^{(k)} - \frac{2(x^{(k)} - x^{(k-1)})f(x^{(k)})}{f(2x^{(k)} - x^{(k-1)}) - f(x^{(k-1)})}. \quad (3.9)$$

ІІ (3.8), (3.9) мають порядок $p \approx 1.62$ і не вимагають обчислення похідної.

Ітераційні методи розв'язання систем нелінійних рівнянь

Знаходження коренів системи n нелінійних рівнянь

$$\vec{f}(\vec{x}) = 0 \quad (3.10)$$

за допомогою ітераційних методів проводиться як і для одного рівняння (3.1).

Метод простої ітерації (МПИ). Спочатку система (3.10) приводиться до еквівалентної

$$\vec{x} = \vec{s}(\vec{x}) \quad (3.11)$$

і ПП проводиться за правилом

$$\vec{x}^{(k+1)} = \vec{s}(\vec{x}^{(k)}), k = 0, 1, \dots \quad (3.12)$$

Теоретичне обґрунтування збіжності ПП (3.10) проводиться в наступній теоремі.

Теор.2.3 Якщо оператор $\vec{s}(\vec{x})$ є *стискаючим* в повному метричному просторі $\mathfrak{R}_n$, то рівняння (3.10) має єдиний розв'язок, який можна знайти методом простої ітерації, причому для похибки $r^{(k)} = \|\vec{x}_* - \vec{x}^{(k)}\|$ має місце наступна умова

$$r^{(k)} \leq \frac{q^k}{1-q} \|\vec{x}^{(1)} - \vec{x}^{(0)}\|,$$

де стала q характеризує «коефіцієнт стиснення» відображення $\vec{s}(\vec{x})$:

$$\|\vec{s}(\vec{x}_1) - \vec{s}(\vec{x}_2)\| \leq q \|\vec{x}_1 - \vec{x}_2\|, \quad 0 < q < 1. \quad (**)$$

Зауваження. Для одновимірного ПП (3.4) умова (**) є умовою Ліпшиця для функції $s(x)$ і у випадку $s(x) \in C^1(\mathfrak{R})$ це означає, що в околі кореня x_* маємо $|s'(x_*)| = q < 1$.

Вектор-функцію $\vec{s}(\vec{x})$ потрібно вибрати так, щоб для матриці $S = [S_{i,j}]_{i,j=1}^n$ з елементами $S_{i,j} = \frac{\partial s_i(x)}{\partial x_j}$ в околі $U_r(\vec{x}_*)$ виконувалась умова $\|S\| \leq q < 1$. Умову закінчення

ПП (3.12) потрібно перевіряти для кожної компоненти:

$$|x_i^{(k+1)} - x_i^{(k)}| \leq \varepsilon, \quad i = \overline{1, n}; \quad 0 < \varepsilon < 1.$$

Метод простої ітерації в поєднанні з методом Зейделя. ПП проводиться за правилом

$$\begin{cases} x_1^{(k+1)} = s_1(x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)}), \\ x_2^{(k+1)} = s_2(x_1^{(k+1)}, x_2^{(k)}, \dots, x_n^{(k)}), \\ \dots \\ x_n^{(k+1)} = s_n(x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{n-1}^{(k+1)}, x_n^{(k)}), \end{cases} \quad (3.13)$$

$$k = 0, 1, \dots$$

Нелінійний метод Зейделя. ПП проводиться за правилом

$$f_i(x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_i^{(k+1)}, x_{i+1}^{(k)}, \dots, x_n^{(k)}) = 0, i = 1, \dots, n; \quad (3.14)$$

$$k = 0, 1, \dots$$

Для (3.13), (3.14) також використовують гібридні методи, коли зовнішні ітерації виконують одним методом, а внутрішні – іншим. Наприклад, зовнішні за Зейделем, а внутрішні – за Ньютоном: використаємо (зовнішній) ПП (3.14), а (внутрішній) для знаходження компоненти $x_i^{(k+1)}$ [1-3].

Метод Ньютона. ПП проводиться за алгоритмом

$$\begin{cases} \vec{x}^{(0)} = \vec{c}_1; \\ \left\{ \begin{aligned} \mathbf{F}(\vec{x}^{(k)}) * \vec{\delta} &= -\vec{f}(\vec{x}^{(k)}), \\ \vec{x}^{(k+1)} &= \vec{x}^{(k)} + \vec{\delta}, \quad k = 0, 1, \dots \end{aligned} \right. \end{cases}, \quad (3.15)$$

де $\mathbf{F}(\vec{x}) = \left(\frac{\partial f_i(\vec{x})}{\partial x_j} \right)_{i,j=1}^n$ – матриця Якобі. Умовою закінчення є

$$\|\vec{x}^{(k+1)} - \vec{x}^{(k)}\| \leq \varepsilon \quad (\text{або } \|\vec{f}(\vec{x}^{(k)})\| \leq \varepsilon), \quad \text{де } 0 < \varepsilon < 1. \quad (3.16)$$

Метод градієнта (метод найшвидшого спуску). ІП проводиться за наступним алгоритмом

$$\begin{cases} \vec{x}^{(0)} = \vec{c}_1; \\ \left\{ \begin{aligned} \vec{z} &= \vec{f}(\vec{x}^{(k)}), \quad \mathbf{J} = \mathbf{F}(\vec{x}^{(k)}), \quad \vec{\delta} = \mathbf{J}' * \vec{z}, \quad \vec{v} = \mathbf{J} * \vec{\delta}, \\ \mu &= \frac{(\vec{z}, \vec{v})}{(\vec{v}, \vec{v})}, \quad \vec{\delta} = \mu * \vec{\delta}, \quad \vec{x}^{(k+1)} = \vec{x}^{(k)} - \vec{\delta}, \quad k = 0, 1, \dots \end{aligned} \right. \end{cases}$$

а умовою закінчення є умова (3.16).

Зауваження. 1. При програмній реалізації методів Ньютона і метода градієнта для зменшення кількості арифметичних операцій бажано обчислення пари $\{f(x^{(k)}), f'(x^{(k)})\}$ або $\{\vec{f}(\vec{x}^{(k)}), \mathbf{F}(\vec{x}^{(k)})\}$ виконувати в одній програмній процедурі.

2. При розв'язанні систем (3.10) існує значна кількість аналогів методу січних (3.8), які реалізуються за наступним алгоритмом :

$$\begin{cases} \vec{x}^{(0)} = \vec{c}_1; \quad \mathbf{B}_0 = \mathbf{F}(\vec{x}^{(0)}), \\ \left\{ \begin{aligned} F_k &= \vec{f}(\vec{x}^{(k)}), \quad \mathbf{B}_k * \vec{\delta} = -F_k, \quad \vec{x}^{(k+1)} = \vec{x}^{(k)} - \vec{\delta}, \\ F_{k+1} &= \vec{f}(\vec{x}^{(k+1)}), \quad \Delta F_k = F_{k+1} - F_k, \\ \mathbf{B}_{k+1} &= \mathbf{B}_k + (\vec{\delta}, \vec{v})^{-1} \cdot F_{k+1} * \vec{v}^T, \quad k = 0, 1, \dots \end{aligned} \right. \end{cases}$$

а умовою закінчення є умова (3.16). Якщо не обчислювати матрицю $\mathbf{F}(\vec{x}^{(0)})$, то $\mathbf{B}_0$ можна взяти одиничну ($\mathbf{E}$) або апроксимувати матрицею $\mathbf{F}(\vec{x}^{(0)}; \vec{x}^{(0)} - \vec{h})$ – узагальненими поділеними різницями. Вектор $\vec{v}$ є параметром даного класу методів, наприклад

- $\vec{v} = \vec{\delta}$ – перший метод Бroyдена;
- $\vec{v} = \mathbf{B}_k^T * \Delta F_k$ – модифікований метод Бroyдена;
- $\vec{v} = \Delta F_k$ – метод Пірсона;
- $\vec{v} = F_{k+1}$ – симетричний метод першого рангу.

3. Аналогічно методу (3.9), для розв'язання систем (3.10) існує *метод Курчатова*, який реалізується наступним алгоритмом :

$$\begin{cases} \vec{x}^{(0)} = \vec{c}_1, \quad \vec{x}^{(1)} = \vec{c}_2; \\ \left\{ \begin{aligned} \mathbf{F}(2\vec{x}^{(k)} - \vec{x}^{(k-1)}; \vec{x}^{(k-1)}) * \vec{\delta} &= -\vec{f}(\vec{x}^{(k)}), \\ \vec{x}^{(k+1)} &= \vec{x}^{(k)} + \vec{\delta}, \quad k = 1, 2, \dots \end{aligned} \right. \end{cases}$$

де $\mathbf{F}(\vec{u}; \vec{w})$ – матриця узагальнених поділених різниць ([1]) першого порядку від функції $\vec{f}(\vec{x}^{(k)})$. Для неї виконується співвідношення $\mathbf{F}(\vec{u}; \vec{w}) * (\vec{u} - \vec{w}) = \vec{f}(\vec{u}) - \vec{f}(\vec{w})$.

Приклади використання деяких ітераційних методів для розв'язання систем нелінійних рівнянь приведено в М-функції *Lе02_1.m*.

В СКМ MATLAB/GNU Octave відсутня пряма реалізація розглянутих ІМ, але є вбудовані функції для розв'язання цього класу задач. Так для числового розв'язання рівняння $f(x) = 0$ наявні наступні функції :

Функції знаходження коренів $f(x) = 0$	
Назва	Призначення
$x = fzero(f, x0)$ $[x, fval, flag, output] = fzero(f, x0, options)$	знаходження нуля функції $f(x)$ поблизу дійсної точки $x0$; якщо $x0=[xl, xr]$, то пошук нуля функції буде на інтервалі $[xl, xr]$, де $f(xl)*f(xr)<0$. Параметри $fval, flag, output, options$ – не обов’язкові. $fval$ – значення функції $f(x)$; $flag$ – код завершення (значення в Octave/MATLAB – див. <i>help fzero</i>); $output$ – структура з інформацією про виконання; $options$ – завдання параметрів управління у вигляді <i>optimset(par1,val1,...)</i> . Найчастіше використовують: ' <i>Display</i> ' – візуалізація ІП, допустимі значення: ' <i>off</i> ' ' <i>iter</i> ' ' <i>notify</i> ' ' <i>TolX</i> ' – точність, значення за угодою = <i>eps</i> .
$X = fsolve(f, X0)$ $[X, fval, flag, output] = fsolve(f, X0, options)$	для скалярної $f(x)$ – знаходження нуля функції, якщо $X0$ – скаляр. У випадку якщо $X0$ – вектор-рядок, то нулі функції при заданих наближеннях. $X0$ – може бути і комплексним. Смисл параметрів – аналогічний функції <i>fzero</i> (за виключенням $X0$). Для отримання детальнішої інформації, використовуйте <i>help fsolve</i> .

Солвер *fsolve* можна використовувати і для розв’язання систем нелінійних рівнянь $\vec{F}(\vec{x}) = \vec{0}$, в цьому випадку синтаксис звертання до цієї функції буде наступним :

$[X, fval, flag, output, Jacob] = fsolve(f, X0, options)$

Більш детальнішу інформацію можна отримати командою *help fsolve*.

В GNU Octave є спеціальні солвери (*bicg, bicgstab, cgs, gmres, pcg, qmr, tfqmr*), які реалізують певні варіанти ІМ з градієнтної групи методів для розв’язання систем нелінійних рівнянь $\vec{F}(\vec{x}) = \vec{0}$.

Для знаходження коренів $f(x) = 0$ | $\vec{F}(\vec{x}) = \vec{0}$ можна використовувати і методи безумовної оптимізації знаходження екстремуму функцій однієї змінної та функцій багатьох змінних. Для цього задачу $f(x) = 0$ | $\vec{F}(\vec{x}) = \vec{0}$ необхідно переформулювати, наприклад, у наступну :

$$\Phi(x) = |f(x)| \rightarrow \min \text{ або } \Phi(x) = |f(x)|^2 \rightarrow \min \quad | \quad \Phi(\vec{x}) = \sum_{i=1}^n |f_i(\vec{x})|^2 \rightarrow \min.$$

Самі методи безумовної оптимізації ми не будемо описувати, але розглянемо використання вбудованих функцій Octave/MATLAB, що реалізують ці методи (тут приводимо тільки синтаксис функцій, а детальну інформацію можна отримати командою *help ім'я_функції*) :

Функції мінімізації функцій і знаходження коренів	
Назва	Призначення
$[x, fval, flag, output] = fminbnd(f, xl, xr, options)$	знаходження мінімуму функції однієї змінної на (xl, xr) . (Призначення і смисл решти параметрів аналогічне солверу <i>fsolve</i>)
$[X, fval, flag, output] = fminsearch(F, X0, options)$	знаходження мінімуму функції декількох змінних в околі точки $X0$
$X = lsqnonlin(F, X0)$ $[X, resnorm, residual, flag] = lsqnonlin(F, X0, LB, UB, options)$ для MATLAB	знаходження $\sum_{i=1}^n f_i^2(\vec{x}) \rightarrow \min, \vec{x} = [x_1, \dots, x_n]$ в околі точки $X0$

Зауваження. Використовуючи функції мінімізації для знаходження коренів скаляр-них рівнянь | систем нелінійних рівнянь потрібно пам’ятати, що при некоректному завданні функції, для якої знаходяться нулі, можна знайти *min*, але це не буде корінь рівняння!

Приклад використання деяких з цих функцій містить наступний М-файл :

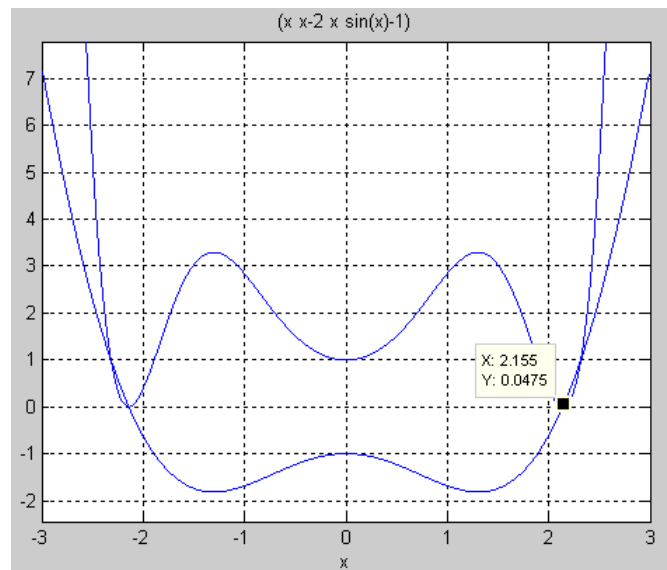
```
%%Le02_2.m
% Знаходження кореня нелінійного рівняння  $f(x)=0$ .
close all, clear all, clc, warning off
```

```

f = @(x) (x.*x-2.*x.*sin(x)-1);
F = @(x) (f(x).^2);
ezplot(F,[-3,3]); hold on; ezplot(f,[-3,3]); grid on
D = input('Інтервал, де знаходиться корінь=');
z = input('Наближення=');
x = fminbnd(F,D(1),D(2));
fprintf('Мінімізація : F(%.6f)=%.6e\n', x, F(x));
x = fzero(f,z); % 1-й варіант виклику
fprintf('Функція fzero: f(%.6f)=%.6e\n', x, f(x));
%з розширеним переліком результатів і виведенням процесу пошуку
[x, fx, ef] = fzero(f,D,optimset('Display','iter'));
fprintf('f(%.6f)=%.6e Код завершення=%2d\n', x, fx, ef);

```

Приведемо результати роботи Le02_2.m :



Інтервал, де знаходиться корінь=[1.8,2.4]

Наближення=2.25

Мінімізація : F(2.145212)=6.674724e-009

Функція fzero: f(2.145196)=0.000000e+000

Func-count	x	f(x)	Procedure
2	1.8	-1.26585	initial
3	2.07285	-0.337401	interpolation
4	2.15397	0.0436669	interpolation
5	2.14467	-0.00257528	interpolation
6	2.14519	-1.76832e-005	interpolation
7	2.1452	9.19673e-011	interpolation
8	2.1452	0	interpolation

Zero found in the interval [1.8, 2.4]

f(2.145196)=0.000000e+000 Код завершення= 1